



Code Encryption for Confidentiality and Execution Integrity down to Control Signals

Theophile Gousselot, Jean-Max Dutertre, Olivier Potin, Jean-Baptiste Rigaud

► To cite this version:

Theophile Gousselot, Jean-Max Dutertre, Olivier Potin, Jean-Baptiste Rigaud. Code Encryption for Confidentiality and Execution Integrity down to Control Signals. IEEE International Symposium on Hardware Oriented Security and Trust (HOST), May 2025, San Jose, United States. hal-04919136

HAL Id: hal-04919136

<https://hal.science/hal-04919136v1>

Submitted on 29 Jan 2025

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.

Public Domain

Code Encryption for Confidentiality and Execution Integrity down to Control Signals

Théophile Gousselot, Jean-Max Dutertre, Olivier Potin, Jean-Baptiste Rigaud
Mines Saint-Etienne, CEA, Leti, Centre CMP, F - 13541 Gardanne France
theophile.gousselot@emse.fr dutertre@emse.fr olivier.potin@emse.fr rigaud@emse.fr

Abstract—Embedded devices face software and physical fault injections to either extract or tamper with code in memory. The code execution and code intellectual property are threatened. Some existing countermeasures provide Control Flow Integrity (CFI) with the confidentiality and integrity of the instructions by chaining all of them through a cryptographic encryption primitive. While tampering with instructions in memory is prevented, fault injection attacks can still target the microarchitecture.

In this work, we introduce a new scheme by chaining the instructions through an encryption with associated control signals. This provides additional authenticity and integrity properties down to the control signals of the microarchitecture's pipeline. The instructions are stored encrypted in memory. At runtime, prior to being executed, the fetched instructions are decrypted depending on the control signals in the pipeline and all the previously decrypted instructions. In case of fault injections, targeting either instructions or control signals, the decryption process fails and generates random instructions, instead of the original ones. This quickly leads to an invalid instruction exception: the fault attack is thwarted.

Our scheme was implemented on FPGA, into the 4-stage pipeline of the RISC-V CV32E40P core, using Ascon for encryption/decryption. When running and validating Embench program executions, we observed at least: a LUT overhead of 28.9%, a reduction of the maximum core frequency of about -29.6%. While patches have to be stored, there is neither clock cycle overhead nor instructions inserted in the code.

Index Terms—fault injection attacks, hardware security, countermeasures, control flow integrity, control signal integrity, chained instruction encryption, Ascon, RISC-V

I. INTRODUCTION

Embedded devices perform a wide range of applications. They integrate efficient cores and sometimes proprietary codes. As critical systems, they can be targeted by attacks aimed at stealing software intellectual property [1]. In addition, tampering with code execution can lead to bypassing security checks, initiating a privilege escalation or leaking confidential data. The code confidentiality and the integrity of its execution are threatened. Attackers can exploit software vulnerabilities [2], [3], or perform physical attacks to threaten the embedded devices, e.g., by extracting device memory. Fault Injection Attacks (FIA) either on instructions in memory or on the device core can break the control flow execution. The FIA targets the hardware layer, but it is the software execution that is altered [4]. Even a single glitch in a core can affect the

execution [5]. Recently, [6] and [7] have enumerated effects of FIA on a microarchitecture core.

Software countermeasures do not always protect against all hardware threats [4] and often result in significant performance degradation. Existing hardware-based countermeasures provide Control Flow Integrity (CFI) against software and physical attacks. Not all CFI solutions are equal. There is a trade-off between the level of security and the performance [8]. Some countermeasures *chained* the legitimate sequences of instructions through a stream cipher [9], [10], [11] or a signature [12], [13]. Therefore, the code integrity is provided in addition to CFI. Countermeasures that encrypt the code also provide confidentiality. Nevertheless, once decoded, instructions are no longer protected. Instead of directly targeting a conditional branch instruction, an attacker can target the control signal [6]. Few countermeasures address the integrity of control signals. Two protect only the static control signal in the decoding stage [14], [11]. One supplements its CFI mechanism by duplicating the static control signals in all core stages [13]. Our contributions to protecting program execution are as follows:

- We introduce a new *4-in-1* scheme that provides code confidentiality as well as integrity and authenticity of the control flow, the instructions and the deterministic control signals from all core stages. Instructions are decrypted depending on the control signals in the core's pipeline and all the previously decrypted instructions. The scheme provides an accurate and *fine-grained* CFI while incurring zero clock cycle penalties.
- We demonstrate the feasibility of modeling control signal propagation in a 4-stage pipeline from a simple analysis of the binary code. We implemented our scheme on an FPGA in the RISC-V CV32E40P core [15], using Ascon cipher.
- We validated and characterized our implementation. We set up a simulation, verification and FPGA design flow environment, capable of running Embench [16] programs and injecting faults.
- Our implementation and our verification environment are open-source and fully available at [17] to incite the community to build up on our work.

We provide a Background in Section II and a State of the art in Section III. We introduce the architecture of our solution in Section IV. The Section V analysis the security of our scheme. Finally, Sections VI and VII details our implementation and its characterization.

The ARSENE project was funded by the “France 2030” government investment plan managed by the French National Research Agency, under the reference “ANR-22-PECY-0004”. We would like to thank the anonymous reviewers for their valuable feedback.

II. BACKGROUND

A. Security Properties provided by Cryptographic Primitives

Various cryptographic primitives may provide distinct security properties like integrity, authenticity and confidentiality of data as summarized in Table I. Contrary to Message Authentication Code (MAC), hash does not involve a key. It cannot provide authenticity and is more suitable for safety rather than security. For the remainder of this paper, the term *signature* refers to hash or MAC as in [12], [13], [18].

Authenticated Encryption (AE) provides confidentiality, authenticity and integrity of data by generating a tag or a MAC. Authenticated Encryption with Associated Data (AEAD) allows authentication of additional unencrypted data, which are not confidential.

TABLE I
SECURITY PROPERTIES PROVIDED BY MAIN CRYPTOGRAPHIC PRIMITIVES.

Property	Hash	MAC	Encryption	AE
Integrity of data	✓	✓		✓
Authenticity of data		✓		✓
Confidentiality of data			✓	✓

B. Program Terminology and Execution

A program is composed of control transfer instructions (e.g., jump, call or conditional branch) and sequential instructions. A common structure in a program is the Basic Block (BB): a set of successive sequential instructions. Only the last instruction of a BB may be a control transfer instruction. There is only one BB entry point: its first instruction, and one exit point: its last instruction. The Control Flow Graph (CFG) defines the valid control transfer between BBs. In this paper, we adopt the RISC-V [19] denomination of control transfer instructions, summarized in Table II.

The successors of an instruction refer to the list of valid instructions that could be executed after it, according to the CFG. For a sequential instruction, there is only one successor: the next instruction in memory.

TABLE II
CONTROL TRANSFER INSTRUCTION CHARACTERISTICS IN RISC-V.

Instruction	Pseudoinstruction	Successor addresses
Direct Jump	<i>jal rd, imm</i>	$pc + imm$
Conditional Branch	<i>b rs1, rs2, imm</i>	$pc + 4 / pc + imm$
Indirect jump	<i>jalr rd, rs1, imm</i>	$rs1 + imm$

Indirect jumps bring two difficulties: (1) they can target a large set of successors, (2) in some cases, exhaustively enumerating the list of each instruction's successors by statically analyzing a program is an unsolved problem [20]. Function calls are direct or indirect jumps while, function returns are always indirect jumps.

In this work, we mainly consider a 4-stage core (*Fetch, Decode, Execute, Write-back*) like the RISC-V CV32E40P. Nevertheless, most concepts could be generalized to different pipeline depths. When a program is executed on a device, instructions are successively fetched from a memory or a

cache. The instruction is then decoded, i.e. transformed into a set of control signals. These control signals are propagated to the rest of the core stages. They drive the flow of data through the data path, e.g., they indicate to the Arithmetic Logic Unit (ALU) the operation to perform.

C. Threat Model

Multiple threats are relevant when protecting a program and its execution. For this work, we consider software and physical attacks, at rest or at runtime. Thus, an attacker can:

- Read and edit all or part of the instructions in memory.
- Microprobe the external core pins, e.g., the instruction memory bus or the Program Counter (PC) as in [21].
- Perform FIA on memory and on the control path of all core stages, as described by [4].

In practice, thanks to a multi-spot laser, [22] have successfully induced four simultaneous and non-contiguous bit sets. [6], [7] have enumerated effects of FIA on a microarchitecture core, and [5] have injected faults on a LEON3 pipeline. Assuming these capabilities, we consider the following scenarios:

- Extract instructions to steal software intellectual property or to discover program vulnerabilities.
- Bypass security checks to initiate a privilege escalation.
- Execute a malicious code or payload.
- Extract a secret key when keyed cryptographic primitives are involved, to forge its own program.

We do not consider side-channel threats, or any tampering with the data: neither in memory nor in the data path of the microarchitecture core. Microprobing the microarchitecture is out-of-the scope of this work.

D. Program Protections

Protecting a program refers to a large set of properties. The main targeted one is the CFI: the property that the control flow respects the CFG, i.e. any CFG-unplanned control transfer is either detected or cannot occur. However, CFI only ensures that the control flow is transferred to one of the multiple possible valid destinations. For example, a function called from one place may return to any of its other call places without breaking CFI. Extending CFI schemes by providing the integrity of call/return executions is highly desirable [23]. To that end, shadow stack [24] should be generalized within CFI schemes.

In this paper, CFI refers to a *fine-grained* CFI [20]. Conversely, a *coarse-grained* CFI approximates the CFG, e.g., by only checking that jumps target the beginning of a BB instead of checking that the BB is a legitimate successor in the CFG. Deliberately relaxing CFI in a *coarse-grained* approach creates major vulnerabilities [8], [25].

Instruction Integrity (II) guarantees that instruction values are not altered until they are decoded (not included in CFI).

Once an instruction is decoded, the Control Signal Integrity (CSI) ensures that control signals in the microarchitecture are not altered. Instruction Confidentiality (IC) guarantees that the instructions are not disclosed (e.g., instructions are not extractable or are encrypted).

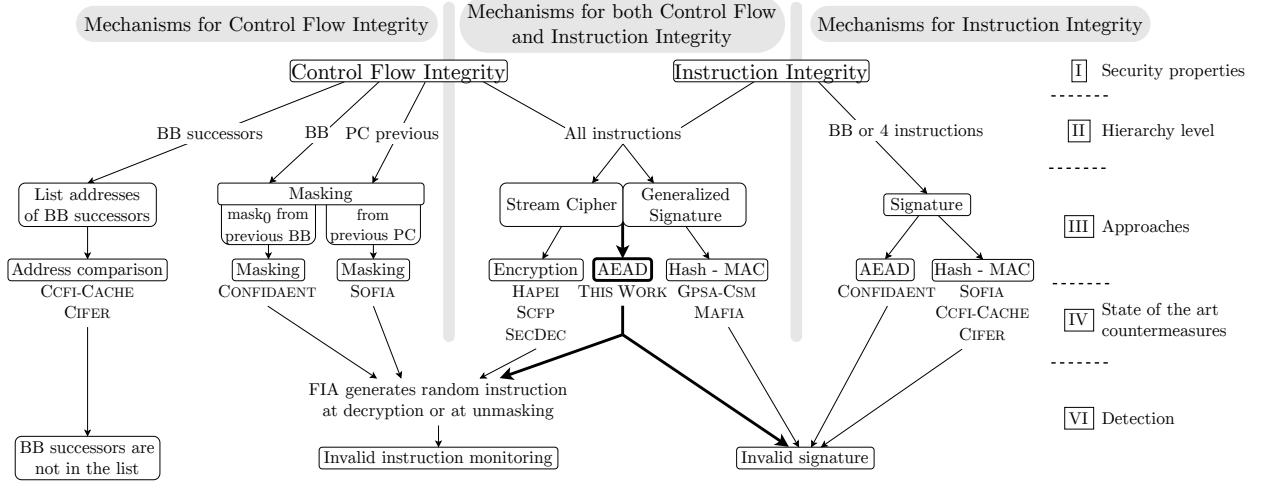


Fig. 1. State-of-the-art mechanisms for Control Flow Integrity and Instruction Integrity.

III. STATE OF THE ART

Most countermeasures ensure the execution integrity of programs against faults in memory or in the instruction bus. We only consider the countermeasures that detect, rather than avoid, FIAs. We focus on those that provide both CFI and II. For comparison, we also consider a software-only solution for CFI against instruction skip fault model [26].

A. State-of-the-Art Mechanisms

In this section, we analyze the existing mechanisms, which are summarized in Fig. 1. Countermeasures providing CFI and II using different mechanisms appear twice: in 1) and in 3).

1) *Mechanisms for Instruction Integrity*: The II results from a signature of BBs for CCFI-CACHE [18], CIFER [14] and SOFIA [27]. At the end of each BB execution the computed signature is compared to a reference one. CONFIDAENT [28] uses Ascon (AEAD) to ensure instruction integrity per group of four. When an instruction is altered, the computed signature will differ from the reference one: *Invalid signature* detection.

2) *Mechanisms for both Control Flow and Instruction Integrity*: Some countermeasures provide CFI and II together with one mechanism. Instead of signing BBs separately, they chain all the instructions through a stream cipher or a signature. Instructions should not be altered (II) and be executed in the chain order (CFI). Otherwise, GPSA-CSM [12] and MAFIA [13] would also generate an *Invalid signature*.

Instead of signing, HAPEI [9], SCFP [10] and SECDEC [11] decrypt instructions at fetching. As soon as an instruction is not the expected one, the stream cipher diverges: its internal state will not follow the expected values. Therefore, random instructions are generated successively. When one is invalid, an exception will occur: *Invalid instruction monitoring*. The latency depends on the density of valid instructions within the Instruction Set Architecture (ISA). This concept was introduced as *Instruction Set Randomization* [29].

3) *Mechanisms for Control Flow Integrity*: Countermeasures ensuring II individually sets up a second mechanism to

provide CFI. For each BB, CCFI-CACHE and CIFER list the addresses of the legitimate successors.

SOFIA masks each instruction according to the address (PC) of the previously executed instruction. CONFIDAENT masks instructions within BBs. The first mask₀ of a BB is stored in all of its predecessor BBs. The mask is then updated at each instruction in the BB. When the control flow is altered, an incorrect mask is used. *Invalid instructions* are generated.

B. Control Flow Integrity Granularity

In all mechanisms for CFI, instructions or BBs are linked to their legitimate successors according to the CFG. There is no difficulty when instructions have only one predecessor or successor. However, the existing mechanisms should also handle cases where the CFG either diverges (multi-successor) or merges (multi-predecessor).

1) Multi-predecessor cases:

a) *Masking from previous PC*: SOFIA creates two entry points for each BBs by expanding BB size. More than 2-predecessor cases are transformed into successive 2-predecessor cases so that the CFG is a binary tree.

b) *Stream Cipher and Generalized Signature*: In HAPEI and SCFP, all the instructions are chained during encryption. However, in Fig. 2-b, it should be possible to decrypt I_2 after either I_1 or I_4 . Since I_2 was encrypted from $State(I_1)$, when I_2 is fetched after I_4 a patch is applied to transform $State(I_4)$ into $State(I_1)$ as illustrated in Fig. 2-c.

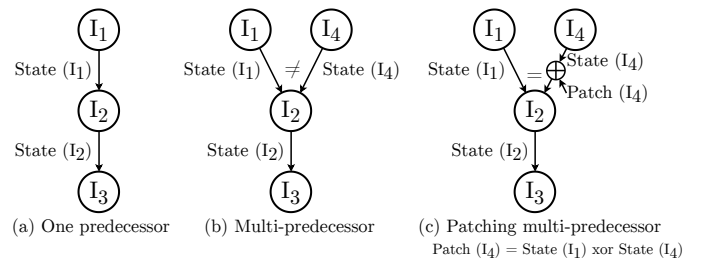


Fig. 2. Control Flow Graph of instructions having one or more predecessors.

GPSA-CSM and MAFIA also patch their signature. We call *patch policy* the specification of:

- How BBs are ordered in the chain.
- Which CFG edges are patched.
- Where patches are stored.
- How patches are accessed.
- When a patch should be applied.

In these countermeasures, BBs are in memory order. Therefore the non-sequential edges are patched. GPSA-CSM and MAFIA store patches in a specific section of memory. They are loaded from custom instructions inserted in the code. SCFP stores each patch next to its control transfer.

They support only one patch per control transfer. Case where several *jalr* target the same destinations as in Fig. 3 are not covered. The CFG is approximated. These solutions only ensure that destinations are in the *equivalence class*. The CFI is not a *fine-grained*, since Jump-Oriented Programming (JOP) or Return-Oriented Programming (ROP) can occur. SECDEC is the more extreme approach, since all the destinations of all the control transfer instructions are in one equivalence class.

MAFIA reduces the use of *equivalence class* by transforming part of the CFG into a binary graph as SOFIA.

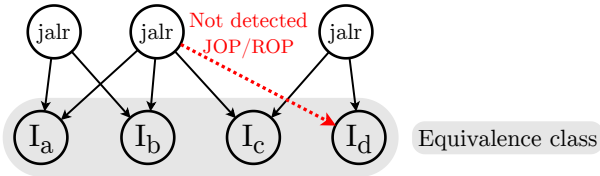


Fig. 3. Control Flow Graph of instructions having more than two successors.

HAPEI proposed patching all but one predecessor edges to ensure a *fine-grained* CFI. Unfortunately, the HAPEI scheme is only implemented on a virtual machine (VM) and is neither described nor validated in a Hardware Description Language (HDL). This approach is not compatible with the one patch per instruction *patch policy*.

2) Multi-successor cases:

a) *List addresses of BB successors:* CIFER and CCFI-CACHE involve an external memory. They associate to each BB a metadata composed of the reference signature and the addresses of legitimate successors. The design of CIFER limits the list to at most two BB successors. The *jalr* are not protected, therefore a *fine-grained* CFI is not provided. CCFI-CACHE can list all *jalr* successors.

b) *Masking from previous BB:* In CONFIDAENT, each BB stores the *mask₀* of its successor BB. The BBs are linked. However, when a BB ends with a *jalr*, all the successor BBs store their own *mask₀* themselves. The BBs are not linked. Therefore, the CFI is not *fine-grained* as JOP attacks are undetected. All *jalr* destinations are in a same *equivalence class*.

C. Control Signal Integrity

The instruction integrity or authenticity does not protect its execution. Execution integrity relies on microarchitecture states, data and control signal integrity. Ensuring data integrity

is a specific research field, although it can be combined with II scheme as in CONFIDAENT.

CIFER and MAFIA sign the control signals instead of the instructions. SECDEC unmask an instruction from the decoded control signals of the previous instruction. However, for the three countermeasures the static program analysis cannot determine the value of signals after the *decode* stage. The values depend on the previously executed instruction (e.g. multi-cycle, data dependency, control transfer). Only the static control signals in the *decode* stage are protected.

MAFIA supplements its generalized signature with a duplication of control signals as [30] for safety. Shadow control signals are propagated in a parallel pipeline, and are compared with their counterparts at each stage.

TABLE III
STATE-OF-THE-ART CONTROL FLOW AND INSTRUCTION INTEGRITY.

Name	Instruction Confidentiality	Control Flow Integrity	Instruction Integrity	Control Signal Integrity	Fine-grained CFI	No clock cycle penalty	No memory overhead
CONFIDAENT [28]	✓	✓	✓				
SOFIA [27]	✓	✓	✓		✓		
CCFI-CACHE [18]		✓	✓		✓	✓	
CIFER [14]		✓	✓	(✓)		✓	
SECDEC [11]		✓	✓	(✓)			
HAPEI [9]	✓	✓	✓		✓		
SCFP [10]	✓	✓	✓				
GPSA-CSM [12]		✓	✓				
MAFIA [13]		✓	✓	✓			
This work	✓	✓	✓	✓	✓	✓	✓

Instruction confidentiality is only provided by encryption of instructions. Table III highlights that no existing countermeasure provides IC, CFI, II and CSI. The CFI is often a *coarse-grained* one. Only MAFIA protects static signals from all stages with a separate module. CIFER and SECDEC protect only static signals in decode (✓). Note that some works ensure CFI specifically against software threats [31], [32], [33].

Most of the countermeasures induce extra clock cycles, caused by the execution of inserted instructions to load patch, verify signatures, transform the CFG to a binary graph or increase BB size (CCFI-CACHE) and break data dependencies (MAFIA) with *nop*. CONFIDAENT as in Instr. Enc. [34] spends cycles to initialize the stream cipher at any BB or group of four instructions. Inserted instructions, the reference signatures, initialization vectors and patches cause memory size overhead in all solutions.

IV. ARCHITECTURE

Our work aims at protecting instructions and their execution against the threat model defined in Section II-C by providing the four security properties:

- Instruction Confidentiality
- Fine-grained Control Flow Integrity & Authenticity
- Instruction Integrity & Authenticity
- Deterministic Control Signal Integrity & Authenticity

Considering existing state-of-the-art countermeasures, we designed the architecture of our solution to address issues ranging from CFI granularity to performance penalties as shown in Table III. In this section, we describe and justify design choices for the architecture of our countermeasure. We then present our specific *patch policy*. Finally, we detail the handling of control signals.

A. Code Encryption with Associated Control Signals

We present a countermeasure that chains the instructions through an encryption with associated control signals:

- *Encrypt then Generate Patches*: After compiling a program, the binary is encrypted. The encryption chains the instructions and links them to control signal values. Patches are generated for the legitimate CFG edges.
- *Decrypt then Execute*: At runtime, the fetched instructions are decrypted depending on the internal state (II). The state depends on all the previously executed instructions (CFI) and control signal values in the pipeline (CSI) as illustrated in Fig. 4. When necessary, the internal state is patched to allow for legitimate Control Flow transfers.

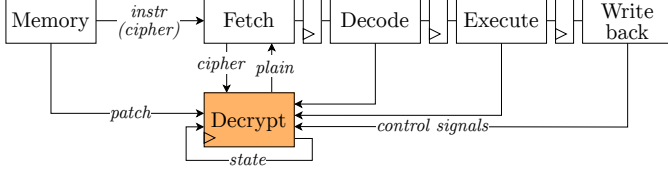


Fig. 4. A module decrypts the encrypted instructions in the *fetch* stage.

1) *Authenticated Encryption rather than a Signature*: Authenticated Encryption (AE) of the instructions ensures Instruction Confidentiality. In addition, AE allows us to associate control signals as *associated data* to provide CSI.

2) *Generalized Chaining of All the Instructions*: All the instructions are chained to provide both CFI and II. Therefore, to support CFG mergers, the internal state of the AE can be patched, as shown in Fig. 5. We proposed a *patch policy* managing patches *in parallel* of instructions, preventing:

- Program or library recompilation.
- Instruction insertion.
- Clock cycle penalty.

3) *Prevent and Detect Fault Injection Attack*: By design, our scheme allows *Invalid instruction monitoring* as shown in Fig. 1. One cycle after an FIA, random instructions are executed instead of the faulted ones. The latency of detection is studied for the RISC-V ISA in Section V-C. Authenticated Encryption also allows for the *Invalid signature* detection.

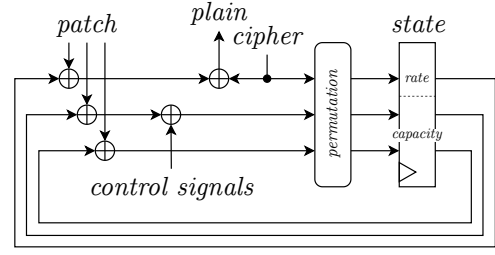


Fig. 5. Implementation of Duplex construction for decrypting instructions.

B. SpongeWrap based on Duplex construction

We select the AEAD mode SpongeWrap based on the MonkeyDuplex construction. The traditional Sponge construction introduced by [35] is based on one *absorbing all input blocks* phase, then one *squeezing all output blocks* phase. The Duplex construction [36] absorbs one input block and generates one output block between two permutation calls. The Duplex construction allows for an efficient implementation of instruction decryption, as only one instruction is fetched per cycle. A variant, the MonkeyDuplex construction introduced by [37], generates the initialization vector from a unique nonce and a secret key to allow for a significant reduction in the number of permutation rounds when decrypting data on the fly.

Only one set of permutation rounds should be implemented, as illustrated in Fig. 5. At each cycle, the encrypted instruction is xored with the rate (most significant bits of the state) to generate the plain instruction.

In [38], the authors enhance the MonkeyDuplex construction by absorbing data on the entire state in a construction called Full-state Keyed Duplex. By adopting this construction, patches and control signals can be xored with the rate and the capacity of the state. SCFP [10] also relied on a Full-state Keyed Duplex construction.

C. Patch Policy

Given a well-defined CFG our *patch policy* provides a fine-grained CFI.

1) *Sequential Encryption for Sequential Instructions*: To reduce the number of patches, the instructions are chained as ordered in memory. In practice, we measured [17] that on average 85.2% of instructions are sequential (i.e., *branches* and all instructions except *jumps*) in Embench programs.

While the instruction following a *jump* in memory should not be executed after the *jump*, a random internal state is used to encrypt such an instruction. This avoids a *jumping over the jump* FIA. As pointed out by HAPFI [9], a random state avoids circular dependencies between states.

2) *Patches for Non-Sequential Control Transfer Edges in the CFG*: To avoid delaying execution, patches are accessed in parallel with instructions. Patches are stored in an external memory, addressed by the PC as defined in Table IV. Conditional branches and direct jumps have only one non-sequential successor at $pc + imm$, as described in Table II. The associated

patch is stored at the address of the *br* or *jal*. Conversely, an indirect jump can have many successors. All patches cannot fit at the address of *jalr*. The patches are stored at the address of the *jalr* successors.

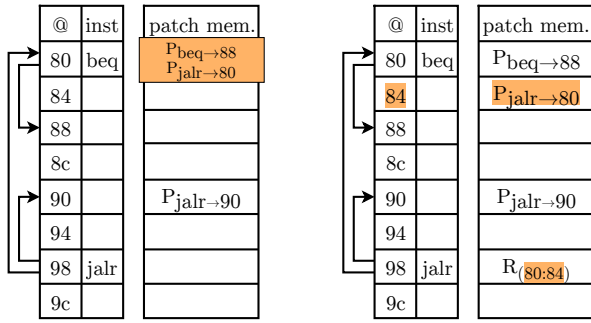
TABLE IV
PATCH ADDRESS OF CONTROL TRANSFER INSTRUCTIONS.

Control transfer instruction	Patch address
Direct Jump	$@_{jal} = P_{jal \rightarrow dest}$
Conditional Branch Taken	$@_{br} = P_{br \rightarrow dest}$
Conditional Branch Not Taken	<i>no need for patch</i>
Indirect jump	$@_{dest} = P_{jalr \rightarrow dest}$
Indirect jump (patch reallocated)	$@_{free} = P_{jalr \rightarrow dest}$

The memory overhead depends on the instruction width and the AEAD state width: $|state| \div |instruction|$. If the state and the instructions have the same size, the memory is doubled.

3) *Reallocation to Resolve Address Conflicts*: There is a conflict when a destination of a *jalr* is a *jal* or *beq*. As illustrated in Fig. 6-a, both patches $P_{beq \rightarrow 88}$ and $P_{jalr \rightarrow 80}$ should be stored at address 80. We developed a *reallocation* method when one successor of a *jalr* is a control transfer instruction. Considering that the address of *jalr* and sequential instructions are free of patches:

- *jalr* patches in conflicts are reallocated at patch-free addresses (e.g., address 84 in Fig. 6-b).
- Reallocations are referenced at the *jalr* address (e.g., if the *jalr* jumps to 80, then data at 98 *readdresses* to 84).



(a) Conflict at address 80 (b) Reallocation for jalr patches

Fig. 6. Reallocation method to avoid patch address conflicts.

If two *jalr* target the same destination, the reallocation method is also applied to one patch. The patch width limits the number of simultaneous *jalr* patches reallocated.

D. Control Signals

Our strategy is consistent with our principles of not stacking countermeasures and not introducing clock cycle overhead. Instead of physically duplicating the control signal pipeline as MAFIA did [13], we associate the expected control signal values at encryption. Thus, the only hardware modification is routing the control signals to the AEAD.

1) *Control Signal Classification*: The solutions for the protection of control signals categorizes them into two types:

- *Static*, which depends only on the instruction in *decode*.
- *Dynamic*, which depends on data or on other instructions.

We are introducing a new classification exemplified in Fig. 7:

- *Deterministic*, which depends only on the instructions (values can be determined by program static analysis).
- *Non-Deterministic*, which depends on data or interrupts.

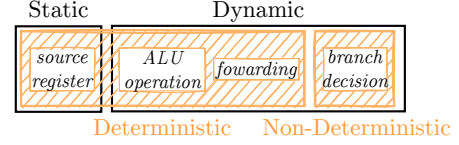


Fig. 7. Static control signals are a subset of the deterministic ones.

2) *Control Signal Propagation*: Without loss of generality, Fig. 8 represents the propagation of control signals through a 4-stage pipeline. Static signals are only the ones directly extracted^① or decoded^② from instruction fields, e.g., *register destination* in the *decode*. However, the propagation of these signals to the *execute* depends on signals from other stages. SECDEC and CIFER limit their protection to static signals in the *decode*. MAFIA protects only the static signals from all stages to avoid having to duplicate combinatory logic.

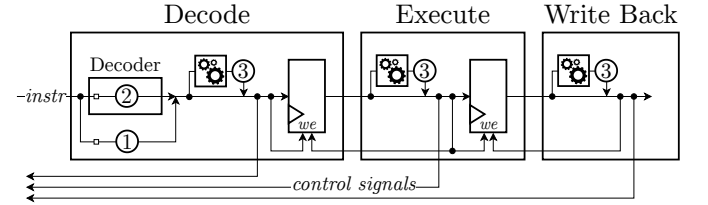


Fig. 8. The origin and propagation of control signals depend on the instruction^{①②} or a combination of other control signals^③.

Some combinational^③ control signals are deterministic. They depend on one or many instructions, e.g., the forwarding signal depends of a data dependency between two instructions. Some other combinational control signals^③ depend on data. They are non-deterministic (e.g., the branch decision). Our scheme is capable of protecting any deterministic control signal from all stages.

3) *Software Model of Control Signal Propagation*: In our scheme, the static program analysis decodes the control signals^{①②} of each instruction. In parallel, the software model infers the values of deterministic control signals^{①②③} from all stages. This allows all the instructions to be encrypted with the associated control signal values. The patches are then computed to support the execution of valid control transfers.

a) *Sequential Execution*: All control signals are propagated for a sequential execution of the instructions. At each instruction, signals are decoded^{①②} and computed^③ according to the signals in the pipeline, as would physically happen in Fig. 8. Our approach is able to model pipeline stalls due

to data dependency or multi-cycle instruction execution. To illustrate, Fig. 9 represents the propagation of one control signal: *alu_operator*. The instruction execution is sequential at (*cyc* 0–6). For example, at (*cyc* 2), the instruction at address 830 is encrypted with control signals from 82c in *decode* (18) and 828 in *execute* (0c). The *alu_operator* is consumed at *execute* and is not propagated to the *write-back*.

cyc	MEM	IF	ID	EX	WB	
0	82C	828	18	03		
1	830	82C	0C	18		
2	834	830	18	0C		
3	838	834	18	18		
4	83C	838	18	18		Comments
5	840	83C	0D	18		branch fetched
6	82C	840	18	0D		branch decoded
7	830	82C	18	03		branch taken
8	834	830	18	03		
	addresses		alu_operator			

Fig. 9. Propagation of a *alu_operator* signal in a 4-stage pipeline.

b) *Control Transfers and Patching*: For each instruction, the control signal values as the internal state used to encrypt are known. Therefore, patches can be computed for each control transfer. Nevertheless, taking a branch or a jump affects some control signal values. Moreover, on a 4-stage core, control transfers are multi-cycle instructions: there are cycles between the fetches of the control transfer and the destination instructions. Some control signals continue to propagate during these cycles. Simultaneously, a branch taken causes a pipeline flush. The patches must be adapted to account for this control transfer-specific propagation of control signals. For example, the instruction 838 is a branch taken to 82c in Fig. 9. At (*cyc* 7) 82c is decrypted with (18 03). At (*cyc* 1), in sequential execution 82c was decrypted with (0c 18). The *alu_operator* default value is (03).

The pipeline has latency. When a second instruction is decrypted after a control transfer, the *decode* stage is like in sequential execution: (18) for 830 at (*cyc* 8) like at (*cyc* 2). Conversely, the *execute* and *write-back* stages still contain residues of the control transfer. Control signals must be patched, (03) must be transformed into (0c) at (*cyc* 8). For a 4-stage pipeline, the control signals from *write-back* used to decrypt the third instruction must be patched. Altogether, patches must be extended with extra bits depending on the number of signals from *execute* and *write-back*: $|signal_{EX}| + 2 \times |signal_{WB}|$. One can generalize for deeper pipelines.

Associating deterministic control signals to the encryption of instruction is not expensive. Physically, control signals are only routed and xored to the AEAD state.

V. SECURITY ANALYSIS

A. Ciphertext-Only Attack Scenario

Considering the classification of Biryukov [39] and the threat model defined in Section II-C, our scheme faces a *Ciphertext-Only Attack* scenario. The attacker only has access to the cipher instruction and the patch memories, neither the plain instructions nor the AEAD internal state.

B. Exploitation of Fault Injections

Our scheme intertwines the execution and decryption of the instructions. Correct execution relies on correct decryption, which relies on unfaulted execution (i.e., unfaulted instructions and control signals). Breaking the CFI by any means or attempting to execute arbitrary code (II, CSI) is prevented by executing random instructions instead, and then is detected. To assess the security effectiveness of our scheme, we adopt the exploitations of FIA enumerated in [40]:

1) *Differential Fault Analysis*: Impossible, because the attacker does not have access to the plain instructions.

2) *Algorithm Specific Attacks*: The selected AEAD algorithm must be highly tested against FIA and should provide the best possible resistance. Its hardware implementation must comply with the specifications, e.g. avoid nonce reuse. A fixed-length ISA does not resort on padding, preventing *Padding Oracle Attack* [41]. The algorithm key must be long enough to not be concerned by *Brute force* threat.

3) *Safe-Error Attacks*: are based on injecting several faults and observing whether they affect the output. By design, an effective FIA on our scheme leaks information by executing random instructions and raising an exception. The internal state at a given time could be discovered. By succeeding to extract instructions and discovering the internal state, an attacker can break the confidentiality of instructions.

C. Probability of Detection when Monitoring Exceptions

The latency of detection when monitoring invalid instruction exceptions depends on the ISA density. To get an order of magnitude, we measured [17] the ratio of valid instructions for the RISC-V ISA by two means: (1) the toolchain exhaustively disassembles the 2^{32} values. We measured a ratio of 0.031 (*rv32im*) and 0.194 (*rv32imc*). (2) The CV32E40P hardware decoder module exhaustively decodes the 2^{32} values. We measure a ratio of 0.046 (*rv32im*) and 0.185 (*rv32imc*), that are close to the previous ones. Fig. 10 shows the probability of not detecting a fault after N instructions decoded. For the *rv32im*, after 58 random instructions executed, the probability of collision is lower than the one of SHA3-256 (2^{-256}).

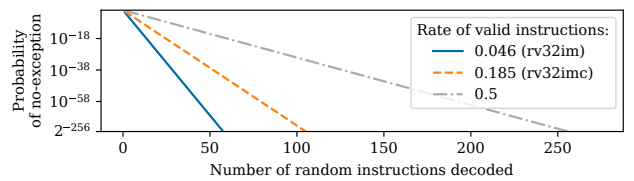


Fig. 10. Probability of generating consecutive instructions that are all valid.

VI. IMPLEMENTATION

In order to demonstrate the feasibility of control signal association, we implement our scheme on the CV32E40P [15], a pipelined 4-stage RISC-V core. Our implementation supports the *rv32im* base instruction set. The deterministic control signal association is modular, one can configure the signals to protect at each stage among all the supported ones. We implement FIA detection only by monitoring the invalid instruction exceptions. The full description and documentation are available at [17].

A. Ascon for Authenticated Encryption with Associated Data

1) *Selection Criterion*: We select Ascon [42] for our scheme implementation. Ascon is the first choice of the CAESAR competition [43] for *Lightweight applications*. Ascon was also selected as a new standard by the NIST Lightweight Cryptography competition [44]. Ascon meets the requirements described in Section V-B: Its implementations has been highly tested against FIA and it operates on a large state of 320 bits.

Ascon is limited to process 2^{64} instructions per key [42]. Considering a frequency of 1 GHz, a program can be safely run continuously for 12 years. This limit depends on the sponge capacity and key sizes [45].

2) *Ascon Adjustments*: According to the *hermetic sponge strategy* [46] the capacity determines the level of security. We safely increase the capacity from 256 to 288 bits, and reduce the rate to match the 32-bit instruction size. Absorption of patch and control signals on the entire state should be valid since we do not generate tag [38].

B. Software Encryption Flow

We developed *python* post-compilation tools to encrypt programs and generate patches according to our *patch policy*.

1) *Control Signals*: All supported control signals of the CV32E40P microarchitecture are characterized: *origin*, *initialized value*, *accessible stages*, *combinatory logic*. A total of 32 control signals (136 bits) are supported in Table. V. A total of 48 other signals (79 bits) were useless considering the hardware configuration of the core.

TABLE V
DETERMINISTIC CONTROL SIGNALS SUPPORTED.

Origin	Decode	Execute	Write-back	examples
Extracted①	13 (24 bits)	8 (17 bits)	4 (6 bits)	<i>alu_operator</i>
Decoded②	2 (12 bits)	1 (6 bits)	1 (6 bits)	<i>rs1, rs2, rd</i>
Combinational③	2 (64 bits)	1 (1 bits)		<i>imm</i>

The decoded② signals depend only on the *funct7*, *funct3* and *opcode* fields of the RISC-V instructions. Once, we built a lookup table that converts an instruction into its decoded control signals. The CV32E40P decoder has exhaustively decoded the 2^{15} values of (*funct7*|0–0|*funct3*|0–0|*opcode*).

The designer can select the control signals to protect at each stage from the set of supported signals.

2) *Sequential Encryption*: To encrypt a program, the control signals to be protected and those on which they depend are modeled, including stalls. The executable sections of the program are parsed, for each instruction:

- The instruction is encrypted by Xoring the rate state.
- The control signals are decoded①② from the instruction.
- The model propagates and computes③ the control signals.
- The state is xored with the associated control signals.
- The current state and control signals are saved in a *fileA*.
- Six permutation rounds are applied to the state.

3) *Patch Generation*: Since we consider a well-defined CFG, the successors of *jalr* that are not returns are listed in a *fileB* from a countermeasure-free program execution. Patches are computed from xored states and are corrected according to control signal values as defined in Section IV-D3b. Patches are generated by parsing the instructions multiple times:

- *First pass*: Lists successors of all instructions except *jalr*.
- *jalr* (except *ret*) successors are read from the *fileB*.
- *ret* successors are determined by a recursive exploration of each *call*.
- States and control signals are loaded from the *fileA*.
- *Second pass*: Generates patches for *jal* and *branch*.
- *Third pass*: Generates patches for *jalr* and recognizes need for reallocation.
- *Fourth pass*: Generates reallocation using patch-free addresses.

C. Hardware Decryption Flow

From the description of the CV32E40P core, our solution:

- Inserts a *Decryption* module into the *fetch* stage as shown in Fig. 4. The decryption module consists of the *Duplex construction* module and a *Patching controller*.
- Routes the control signals to the *Decryption* module.
- Inserts a patch memory addressed by the PC.

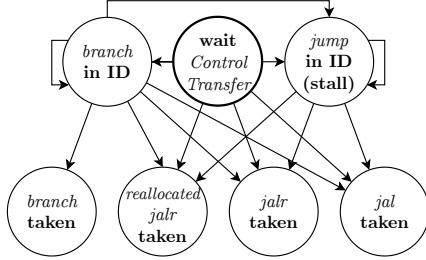
1) *Duplex Construction Module*: The state register and its permutation rounds are described as in Fig. 5. The initial state (precomputed from the secret key) is directly loaded into the state register. Other key management modes are possible depending on the application. Ascon initialization stage may be requested. Since the module is deep in the core, the decryption is synchronized with the core. Namely, when the *fetch* is stalled there is no decryption and the state register is not updated.

The core fetches at most one instruction per clock cycle. According to the Ascon specifications [42], 6 Permutation Units (PU) must be implemented. To reduce the area overhead, either 6, 3, 2 or 1 PUs can be physically implemented. In the latter cases, we set another clock to iterate over the 3, 2 or 1 PUs. This second clock is respectively 2, 3 or 6 times faster than the core clock, to decrypt up to one instruction per cycle.

Between the patch memory and the logical xor with the state in the *Duplex construction* module, there are two *shift registers* to handle reallocation and cases where a *jump* follows a *branch* in memory. A *branch* is taken at *execute* while a *jump* is taken at *decode*. In this case, when the branch

decision is computed, both patches of *branch* and *jal* are ready to be xored.

2) *Patching Controller Module*: Patch management should not incur clock cycle penalties. We design a Finite-State Machine (FSM), shown in Fig. 11, to manage successive control transfers even when a stall occurs, without losing clock cycles to fetch the patch. The FSM keeps pace with the execution. For example, any case where a control transfer follows a branch in memory is supported. Any multi-cycle instruction that causes a stall when a *jump* is in the *decode* is also supported. The FSM mainly selects the patch to apply from the *shift registers*. The FSM also handles the *jalr* reallocation, by selecting the address to read in the patch memory.



Every state can reach "wait Control Transfer" except "jump in ID (stall)".

Fig. 11. The Finite-State Machine manages the patching of control transfers.

VII. IMPLEMENTATION CHARACTERIZATION

We validated our implementation by running encrypted programs and observing their well execution and that FIA are detected. We characterized our implementation by measuring memory overhead, maximum frequency reduction, utilization overhead, and confirming the zero clock cycle overhead of our scheme. We did not observe any significant increase in compilation time due to our post-compilation tools.

A. Verification Environment

We developed the *core-v-verif-fpga* environment [17] to execute encrypted programs from simulations to FPGA.

1) *Environment Features*: The environment can build unprotected and protected versions of CV32E40P core and memories. The protected versions vary according to the configuration of the control signals to protect and the number of physically implemented PUs. All Embench [16] programs can be compiled and executed on any version. The environment allows:

- Cycle-accurate simulation and verification (Verilator)
- FPGA design flow (Vivado).
- Timing simulation and verification (Modelsim)
- Execution and validation on FPGA board.
- FIA on FPGA (memory only) and on simulations.

We program bitstreams on an *Artix-7 200T* in a *Nexys Video board*. Memories are synthesized in *Block RAMs* with single-cycle latency. A *Mixed-Mode Clock Manager* generates the clocks for the core and for Ascon. Since our scheme only consumes binary, no libraries were recompiled.

2) *State-of-the-art Verification Environment*: Most of the state-of-the-art solutions introduced in Section III are validated only on logical cycle-accurate simulations, as summarized in Table VI. Only SOFIA and CIFER explicitly achieve post-synthesis simulations. Only HAPEI provides source code.

TABLE VI
IMPLEMENTATION OF THE STATE-OF-THE-ART SOLUTIONS.

Name	Available	HDL	Synth.	Validation	FIA
INSTR. ENC. [34]		✓	FPGA	logic	
CONFIDAENT [28]		✓		logic	
SOFIA [27]		✓	FPGA	FPGA	
CCFI-CACHE [18]		✓	FPGA	logic	✓
CIFER [14]		✓	FPGA	FPGA	✓
SOFT-ONLY [26]	✓	✓		logic	✓
SECDEC [11]		✓	ASIC	logic	✓
HAPEI [9]	✓			VM	✓
SCFP [10]		✓	ASIC	logic	
GPSA-CSM [12]		✓	ASIC	logic	
MAFIA [13]		✓	ASIC	logic	
This work	✓	✓	FPGA	FPGA	✓

3) *Encrypted Execution Verification*: For each simulation on a protected core, the PC and the instruction in the *fetch* are compared at each cycle with those of an unprotected simulation. The 22 Embench programs were successfully executed on all versions, including the association of all control signals presented in Section VI-B1. Complex cases were encountered, such as stall due to a data dependency between a *load* and a *branch* taken to a *jalr*. Only two programs *qrduino* and *picojpeg* exceed the limit of 12 destinations of a reallocated *jalr* introduced in Section IV-C3. This limit is not very restrictive. For example, we succeed to make these two programs compliant with our *patch policy* by inserting 3 and 2 *nop* to remove the specific reallocation. For both programs, the code size and clock cycle overheads are less than 0.06%.

B. Characterization

Our scheme provides the four security properties by involving only one AEAD algorithm. The raw overheads must be broken down to understand whether they come from the scheme, its *patch policy*, its implementation optimizations or the selected algorithm. Clearly, the 320 bits of Ascon state cause significant overheads.

1) Overhead Breakdown Analysis:

a) *Utilization and Frequency*: Table VII provides utilization and maximum frequency depending on the number of PUs physically implemented. One round costs around 360 LUTs (1 or 2 LUTs per bit in state). The *two shift registers*, the *Block RAM* output register, and the state register (doubled if different clocks) cause 320 Flip-Flops each. The critical path is from the patch memory to the last permutation round of the first Ascon clock cycle. The association of control signals adds a low utilization cost. For example, a configuration with 21 bits of control signals (6 PUs+CS) is shown in Table VII.

b) *Patches*: As expected, there is one patch per *jal* and *branch* instruction and as many as successors for *jalr*.

TABLE VII
MEASURED UTILIZATION AND MAXIMUM FREQUENCY.

Versions	Max. Freq. (MHz)	# LUT	# FF
Unprotected	67.45	4298	2064
6 PUs+CS	37.0 (-45.1 %)	7395 (+72.1 %)	3435 (+66.4 %)
6 PUs	47.5 (-29.6 %)	7218 (+67.9 %)	3402 (+64.8 %)
3 PUs	34.0 (-49.6 %)	6274 (+46.0 %)	3733 (+80.9 %)
2 PUs	30.0 (-55.5 %)	5904 (+37.4 %)	3733 (+80.9 %)
1 PU	19.0 (-71.8 %)	5541 (+28.9 %)	3733 (+80.9 %)

Table VIII summarizes the number of patches and reallocations for a subset of programs, the full table is available at [17]. Reallocations are required for one tenth of *jalr*. On average 74.8% lines of patch memory are empty.

TABLE VIII
NUMBER OF PATCHES AND REALLOCATIONS IN EMBENCH PROGRAMS .

Program	#instr.	#br	#jal	#jalr	#patch	#reallo.
aha-mont	2354	247	108	249	603 (25.6 %)	23 (1.0 %)
crc32	3359	340	148	390	885 (26.3 %)	30 (0.9 %)
cubic	12124	838	186	1342	2564 (21.1 %)	46 (0.4 %)
...						
wikisort	7147	594	230	816	1737 (24.3 %)	62 (0.9 %)
Average	4404	392	146	495	1076 (24.4 %)	34 (0.8 %)

2) *Enhancement and Comparison with the State of the art:* The Table IX is only intended to show overhead trends. The test environment, the technology, the ISA, the core and the programs executed were all different. Also, very few solutions provide the four security properties with a fine-grained CFI.

TABLE IX
PERCENTAGE OVERHEADS FOR STATE-OF-THE-ART SOLUTIONS.

Name	Memory† (%)	Cycles (%)	Freq. (%)	Utilization (%)
INSTR. ENC. [34]	2–38	129–293*	0	26 ALM
CONFIDAENT [28]	116–218*	151–276*		
SOFIA [27]	110–437	36–438	–23	12 LUT
CCFI-CACHE [18]	118–160*	2–63		11 LUT 9 FF
CIFER [14]	16–68	0	0	35–55 Slice
SOFT-ONLY [26]	9–3550*	92–3100*	0	null
SECDEC [11]	1–2	1–2		4–17 GE*
HAPEI [9]	123–507*			
SCFP [10]	15–26	4–15		47 GE
GPSA-CSM [12]	88–118*	2–71*		6 GE
MAFIA [13]	7–55	3–44		7–24 GE
This work	100/32× state	0		
(Ascon)	1047	0	–30	29 LUT 81 FF
(Prince)	247	0		

When available the *best–worst* overheads are given.

* : not directly given, we computed from other raw data in the article.

†: Inserted instructions, signatures and patches are included. The memory baseline is sometimes the size of program instructions and data.

LUT: Lookup Table (FPGA - Xilinx: Vivado) FF: Flip Flop (FPGA - Xilinx: Vivado)
GE: Gate Equivalent (ASIC) ALM: Adaptive Logic Module (FPGA - Altera: Quartus)

a) *Memory Size:* Inserted instructions (i.e. *nop*, *load*, *jump*), tags and patches cause memory size overhead. Our scheme inserts zero instruction, all of our overhead is due to our *patch policy*. The patch memory is 320 bits wide. Implementing our scheme with the 64-bit state Prince as used by SOFIA, SCFP and MAFIA would reduce the memory

expansion by a factor of five. Since three quarters of the patch memory is empty, a lookup table could readdress patches to a full but smaller memory. Considering a program having 2^{16} instructions the memory sizes would change from 320×2^{16} to $14 \times 2^{16} + 320 \times 2^{14}$ (3.4 times smaller).

b) *Clock Cycles:* The interest of our *patch policy* is to provide a zero clock cycle penalty, only Cifer does as well.

c) *Maximum Frequency and Utilization:* As discussed, few solutions have been implemented on FPGA or ASIC. The penalties are mainly due to the implemented cryptographic primitive. The overheads depend primarily on the maximum frequency and utilization of the unprotected core.

C. Experimental Validation

We injected memory faults by editing the memory contents in simulations and on the FPGA. We inject control signal faults only in simulations, as illustrated in Fig. 12. Each time, random instructions were generated by Ascon decryption, and the attack was detected in a few cycles. Full chronograms are available at [17].

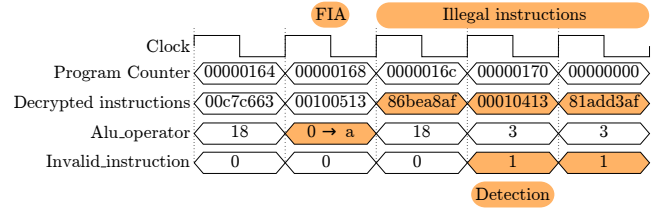


Fig. 12. Chronogram of a detected FIA on *alu_operator* signal.

VIII. CONCLUSION

We associate the microarchitecture control signals to a chain encryption of instructions to provide code confidentiality and execution integrity down to the control signals. Our scheme does not stack countermeasures. A single hardware cryptographic primitive provides instruction confidentiality as well as the integrity and the authenticity of the control flow, the instructions and the deterministic control signals. Attacks on memory, external bus, and deterministic control signals from all stages are covered. By design, faulted instructions cannot be executed. We strongly recommend supplementing our scheme with a shadow call stack, and a data protection mechanism. In a non-stacking approach, our scheme can associate the address from a shadow call stack to ensure return execution integrity.

The implementation of our scheme demonstrates its feasibility on a 4-stage, in order core. The compilation chain remains untouched. The proposed patch policy avoids clock cycle penalty at the cost of additional memory. Ascon is reliable, however its wide state causes higher memory overhead than the commonly used Prince. Our implementation and our verification environment are open-source and available at [17].

We plan to inject electromagnetic faults on FPGA to assess the effective control signal integrity. One perspective of this work is to investigate methods to reduce patch size: chaining BBs with a part of the state only or implementing a primitive with a smaller state. We intend to support caches, interrupts and active branch predictors by saving the internal state.

REFERENCES

- [1] S. Skorobogatov, "Physical attacks and tamper resistance," in *Introduction to Hardware Security and Trust*. Springer, 2012, pp. 143–173.
- [2] R. Roemer, E. Buchanan, H. Shacham, and S. Savage, "Return-oriented programming: Systems, languages, and applications," *ACM Transactions on Information and System Security (TISSEC)*, vol. 15, no. 1, pp. 1–34, 2012.
- [3] T. Bletsch, X. Jiang, V. W. Freeh, and Z. Liang, "Jump-oriented programming: a new class of code-reuse attack," in *Proceedings of the 6th ACM symposium on information, computer and communications security*, 2011, pp. 30–40.
- [4] B. Yuce, P. Schaumont, and M. Witteman, "Fault attacks on secure embedded software: Threats, design, and evaluation," *Journal of Hardware and Systems Security*, vol. 2, pp. 111–130, 2018.
- [5] B. Yuce, N. F. Ghalaty, H. Santapuri, C. Deshpande, C. Patrick, and P. Schaumont, "Software fault resistance is futile: Effective single-glitch attacks," in *2016 Workshop on Fault Diagnosis and Tolerance in Cryptography (FDTC)*. IEEE, 2016, pp. 47–58.
- [6] J. Laurent, V. Beroulle, C. Deleuze, F. Pebay-Peyroula, and A. Papadimitriou, "Cross-layer analysis of software fault models and countermeasures against hardware fault attacks in a risc-v processor," *Microprocessors and Microsystems*, vol. 71, p. 102862, 2019.
- [7] S. Tollec, M. Asavao, D. Couroussé, K. Heydemann, and M. Jan, "Exploration of fault effects on formal risc-v microarchitecture models," in *2022 Workshop on Fault Detection and Tolerance in Cryptography (FDTC)*. IEEE, 2022, pp. 73–83.
- [8] N. Burrow, S. A. Carr, J. Nash, P. Larsen, M. Franz, S. Brunthaler, and M. Payer, "Control-flow integrity: Precision, security, and performance," *ACM Computing Surveys (CSUR)*, vol. 50, no. 1, pp. 1–33, 2017.
- [9] R. Lashermes, H. Le Boudier, and G. Thomas, "Hardware-assisted program execution integrity: Hapei," in *Secure IT Systems: 23rd Nordic Conference, NordSec 2018, Oslo, Norway, November 28-30, 2018, Proceedings 23*. Springer, 2018, pp. 405–420.
- [10] M. Werner, T. Unterluggauer, D. Schaffner, and S. Mangard, "Sponge-based control-flow protection for iot devices," in *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2018, pp. 214–226.
- [11] G. Leplus, O. Savry, and L. Bossuet, "Secdec: Secure decode stage thanks to masking of instructions with the generated signals," in *2022 25th Euromicro Conference on Digital System Design (DSD)*. IEEE, 2022, pp. 556–563.
- [12] M. Werner, E. Wenger, and S. Mangard, "Protecting the control flow of embedded processors against fault attacks," in *Smart Card Research and Advanced Applications: 14th International Conference, CARDIS 2015, Bochum, Germany, November 4-6, 2015. Revised Selected Papers 14*. Springer, 2016, pp. 161–176.
- [13] T. Chamelot, D. Couroussé, and K. Heydemann, "Mafia: Protecting the microarchitecture of embedded systems against fault injection attacks," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2023.
- [14] A. Zgheib, O. Potin, J.-B. Rigaud, and J.-M. Dutertre, "Cifer: Code integrity and control flow verification for programs executed on a risc-v core," in *2023 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. IEEE, 2023, pp. 100–110.
- [15] "Openhw group, cv32e40p, a 4-stage, 32-bit, risc-v core," 2022, available at <https://github.com/openhwgroup/cv32e40p>.
- [16] Embench, "Embench: Open benchmarks for embedded platforms <https://github.com/embench/embench-iot>," 2022.
- [17] T. Gousselot. (2024) Github repository containing the open-source implementation and the verification environment. Available at <https://github.com/theophile-gousselot/execution-integrity-down-to-control-signals>.
- [18] J.-L. Danger, A. Facon, S. Guille, K. Heydemann, U. Kühne, A. S. Merabet, and M. Timbert, "Ccfi-cache: A transparent and flexible hardware protection for code and control-flow integrity," in *2018 21st Euromicro Conference on Digital System Design (DSD)*. IEEE, 2018, pp. 529–536.
- [19] A. Waterman, Y. Lee, D. A. Patterson, and K. Asanovic, "The risc-v instruction set manual, volume i: User-level isa," *CS Division, EECE Department, University of California, Berkeley*, p. 28, 2014.
- [20] R. De Clercq and I. Verbauwhede, "A survey of hardware-based control flow integrity (cfi)," *arXiv preprint arXiv:1706.07257*, 2017.
- [21] C. Helfmeier, D. Nedospasov, C. Tarnovsky, J. S. Krissler, C. Boit, and J.-P. Seifert, "Breaking and entering through the silicon," in *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*, 2013, pp. 733–744.
- [22] B. Colombarier, P. Grandamme, J. Vernay, É. Chanavat, L. Bossuet, L. de Launanié, and B. Chassagne, "Multi-spot laser fault injection setup: New possibilities for fault injection attacks," in *International Conference on Smart Card Research and Advanced Applications*. Springer, 2021, pp. 151–166.
- [23] N. Carlini, A. Barresi, M. Payer, D. Wagner, and T. R. Gross, "Control-flow bending: On the effectiveness of control-flow integrity," in *24th USENIX Security Symposium (USENIX Security 15)*, 2015, pp. 161–176.
- [24] T. H. Dang, P. Maniatis, and D. Wagner, "The performance cost of shadow stacks and stack canaries," in *Proceedings of the 10th ACM Symposium on Information, Computer and Communications Security*, 2015, pp. 555–566.
- [25] L. Davi, A.-R. Sadeghi, D. Lehmann, and F. Monrose, "Stitching the gadgets: On the ineffectiveness of coarse-grained control-flow integrity protection," in *23rd USENIX Security Symposium (USENIX Security 14)*, 2014, pp. 401–416.
- [26] F. Bonnal, V. Dupakis, O. Potin, and J.-M. Dutertre, "Software-only control-flow integrity against fault injection attacks," in *2023 26th Euromicro Conference on Digital System Design (DSD)*. IEEE, 2023, pp. 269–277.
- [27] R. De Clercq, J. Götzfried, D. Übler, P. Maene, and I. Verbauwhede, "Sofia: Software and control flow integrity architecture," *Computers & Security*, vol. 68, pp. 16–35, 2017.
- [28] O. Savry, M. El-Majhi, and T. Hiscock, "Confidaent: Control flow protection with instruction and data authenticated encryption," in *2020 23rd Euromicro Conference on Digital System Design (DSD)*. IEEE, 2020, pp. 246–253.
- [29] G. S. Kc, A. D. Keromytis, and V. Prevelakis, "Countering code-injection attacks with instruction-set randomization," in *Proceedings of the 10th ACM conference on Computer and communications security*, 2003, pp. 272–280.
- [30] S. Kim and A. K. Somani, "On-line integrity monitoring of microprocessor control logic," in *Proceedings 2001 IEEE International Conference on Computer Design: VLSI in Computers and Processors. ICCD 2001*. IEEE, 2001, pp. 314–319.
- [31] M. Abadi, M. Budiu, U. Erlingsson, and J. Ligatti, "Control-flow integrity principles, implementations, and applications," *ACM Transactions on Information and System Security (TISSEC)*, vol. 13, no. 1, pp. 1–40, 2009.
- [32] M. Castro, M. Costa, and T. Harris, "Securing software by enforcing data-flow integrity," in *OSDI*, vol. 6, 2006, pp. 147–160.
- [33] M. Gallagher, L. Biernacki, S. Chen, Z. B. Aweke, S. F. Yitbarek, M. T. Aga, A. Harris, Z. Xu, B. Kasikci, V. Bertacco *et al.*, "Morpheus: A vulnerability-tolerant secure architecture based on ensembles of moving target defenses with churn," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2019, pp. 469–484.
- [34] T. Hiscock, O. Savry, and L. Goubin, "Lightweight instruction-level encryption for embedded processors using stream ciphers," *Microprocessors and Microsystems*, vol. 64, pp. 43–52, 2019.
- [35] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche, "Sponge functions," in *ECRYPT hash workshop*, vol. 2007, no. 9, 2007.
- [36] —, "Duplexing the sponge: single-pass authenticated encryption and other applications," in *Selected Areas in Cryptography: 18th International Workshop, SAC 2011, Toronto, ON, Canada, August 11-12, 2011, Revised Selected Papers 18*. Springer, 2012, pp. 320–337.
- [37] —, "Permutation-based encryption, authentication and authenticated encryption," *Directions in Authenticated Ciphers*, pp. 159–170, 2012.
- [38] B. Mennink, R. Reyhanitabar, and D. Vízár, "Security of full-state keyed sponge and duplex: Applications to authenticated encryption," in *Advances in Cryptology—ASIACRYPT 2015: 21st International Conference on the Theory and Application of Cryptology and Information Security, Auckland, New Zealand, November 29–December 3, 2015, Proceedings, Part II 21*. Springer, 2015, pp. 465–489.
- [39] A. Biryukov, "Des-x (or desx)." 2011.
- [40] D. Karaklajić, J.-M. Schmidt, and I. Verbauwhede, "Hardware designer's guide to fault attacks," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 21, no. 12, pp. 2295–2306, 2013.

- [41] R. Bardou, R. Focardi, Y. Kawamoto, L. Simionato, G. Steel, and J.-K. Tsay, "Efficient padding oracle attacks on cryptographic hardware," in *Annual Cryptology Conference*. Springer, 2012, pp. 608–625.
- [42] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schl  ffer, "Ascon," *Submission to the CAESAR competition*: <http://ascon.iaik.tugraz.at>, 2014.
- [43] Bernstein, "Caesar: Cryptographic competition for authenticated encryption: Security, applicability, and robustness: Final portfolio <https://competitions.cr.yp.to/caesar.html>," 2019.
- [44] M. S. Turan, M. S. Turan, K. McKay, D. Chang, L. E. Bassham, J. Kang, N. D. Waller, J. M. Kelsey, and D. Hong, *Status report on the final round of the NIST lightweight cryptography standardization process*. US Department of Commerce, National Institute of Standards and Technology, 2023.
- [45] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche, "On the security of the keyed sponge construction," in *Symmetric Key Encryption Workshop*, vol. 2011, 2011.
- [46] B. Guido, D. Joan, P. Micha  l, and V. Gilles, "Cryptographic sponge functions," 2011.